

The bitcoin mining process has a ton of relevant information that is being constantly updated with each block mined. Usually, this kind of data is handled automatically so there is no need to check it constantly, but when learning how everything works it is useful as an exercise to have (or in my case make) a dashboard that displays it all.

All the information displayed comes from various API calls, or at least it is calculated from those. The API in question is “<https://mempool.space/api>”, and all its documentation can be found in “<https://mempool.space/docs/api/rest>”.

The dashboard itself (AKA how the information is displayed) is built with streamlit, and it is mostly vibecoded. This explains why it runs poorly and why it kind of freezes every 30 seconds, as in the process of making it update the displayed data in real time the only solution I found was to reload the entire dashboard (including API calls) passed the refresh time.

The dashboard is divided into four modules.

The first one show various data related with how difficult is to mine the blocks/ how much work is being done to mine them.

On top of all there is the difficulty, a number that indirectly represents what fractions of all hashes that are valid. For a hash to be accepted it needs to have a higher difficulty score than the one you can find here. To make it easier to visualize, if you treat the hash as a regular number, the lower the number the higher the difficulty score it has, so the more zeros it has at the beginning the better. The second value in the module is in fact this amount of zeros, and it is calculated with the logarithm in base 2 of the difficulty plus 32 and divided between 4 (as the hashes are being displayed in hexadecimal).

Bellow those two values there is a graph that shows how much time it took to mine each the last n block, with n being an adjustable number that starts at 5. This was supposed to use the batch function in the API, but after struggling with the code for a while I realized that it simply didn't work exactly as the documentation said, so I had to find another way. This way is calling in succession all the block needed thanks to the previous block found in each of them starting from the block at the tip and seeing the differences in their time stamps. This explains why it takes so long to process, as it has to make an API calls one after the other.

The last thing in the module is the hash rate, or the amount of hashes being tried per second. It is just looked up in the API.

The second module just contains a tool for checking that a specific hash is valid. You can just enter any previous hash, but there is a button that automatically enters the last one. It first fetches all the data from the corresponding block needed for the proof of work. Then it construct the header, showing with a legend which bits come from each data. Then it does a sha256 to it and compares it with the target to confirm it is smaller than the limit.

The third module is a simple histogram of how the difficulty has changed over time. These changes occur every 2016 blocks mined (roughly 2 weeks) and are intended to adapt the blockchain to the hash rate and approximate that mining time to 2 weeks (make it harder if going too fast, and easier if slow). It can be set to show a period of time of 1, 3, 6 months, 1, 2, 3 years or just from the beginning. Sadly, the “all” option doesn’t seem to work despite of what the API documentation said, but I also found that the data isn’t sent in the format it says it is, so I don’t know.

And the last module is an AI predictor of the difficulty adjustments. You can select a period of time like in the module 3 and use that data to train a model that will try to predict the following year of adjustments. It is trained with the prophet model of the library of the same name. Obviously, the more data you feed it, the better the results will be. I only need to point out that this is the module the constant refreshing breaks the most.

Anyway, with all these modules done (and specially after making this report, I must admit) my understanding of all the concepts related with blockchain have solidified a great deal. For example I had to learn how to calculate the leading zero threshold based on the difficulty.

Sadly I hadn’t time to do any extra modules, as I left them for the last minute and when I got to it realized there were still a bunch of things to fix in the other four modules done a week before, plus this report as I had forgotten it was a thing.

I think I have updated the git hub correctly, although I have to admit that my commit policy could be better, as I wait too much between one commit and the next one (plus there was one I forgot to push for an entire week for no reason).